Full-Stack Developer (Intern)
Practical Assignment

Assignment: Develop a demo log management system that supports multiple data sources and can be deployed both as a hardware appliance (on a single machine/VM) and as SaaS/Cloud (with a URL accessible for external testing).

Submission deadline and demo interview: Within 10 days after viewing this assignment. The student is responsible for scheduling the appointment.

Work format: Individual

Technology: Candidates may choose their own tech stack. Open-source and cloud-native technologies are recommended.

---

1) Objectives

1. Assess full-stack capabilities: architecture design, Backend/API, Frontend/UI, data pipelines, and DevOps/deployment.
2. Assess the handling of event/log data from multiple sources, including normalization, indexing, search, visualization, and alerting.
3. Assess understanding of security, including AuthN/AuthZ, TLS, and multi-tenant customer data isolation.

---

2) Scope & Must-have Requirements

2.1 Required data sources (at least 4 real, working sources; the remainder may use samples/simulators)

- Firewall / Network (e.g., Syslog UDP/TCP 514 or HTTP ingest)
- API (receive JSON via REST/HTTP POST)
- CrowdStrike (synthetic sample JSON/CSV is permitted)
- AWS (e.g., CloudTrail/ALB/NLB; sample files may be used)
- Microsoft 365 (Unified Audit Log; sample JSON may be used)
- Microsoft AD/Windows Security (Event IDs 4624/4625, etc.; samples may be used)

Note: Real connections to every source are not required. At least four sources must actually be able to send data into the system (for example, Syslog + HTTP API + JSON batch file + a simulator script). All sources must be normalized into a centralized schema before storage.

2.2 Minimum functional requirements

- Ingestion: Accept logs in multiple formats (Syslog / HTTP JSON / file batch) and support at least two protocols.
- Normalization: Convert every log type into a common schema (see Section 3 for an example).
- Storage & Query: Store data in a searchable system (e.g., OpenSearch, ClickHouse, PostgreSQL + GIN, etc.).
- Dashboard: Provide a UI summary with Top IP/User/Event Type, a timeline, and filters for time range and tenant.

- Alert (minimum 1 rule): Define a simple condition (e.g., repeated failed logins from the same IP within five minutes), then show it on an Alert page or send it via webhook/email.
- AuthN/AuthZ: Provide at least two roles (Admin/Viewer) and support tenant data isolation through a parameter, header, or claim.
- Deployment in 2 modes:
- Appliance: Run on a single machine or VM (Docker Compose recommended).
- SaaS/Cloud: Run on a public cloud (e.g., VM/container) with a URL accessible to the reviewers.
- TLS: Enable HTTPS at least in SaaS mode. A self-signed certificate is acceptable if the setup is clearly documented.
- Retention: Retain data for at least seven days, using deletion, rollover, or partitioning.

2.3 Nice-to-have (optional)

- True multi-tenancy (separate indexes/tables) and field/tenant-level RBAC.
- Enrichment during ingestion (e.g., reverse DNS, GeoIP).
- CI/CD (GitHub Actions, etc.) and IaC (Terraform/Helm).
- Basic unit/integration tests.

---

3) Recommended Common Schema

The schema may be adjusted, but it should cover the key fields required for search:

```
@timestamp (RFC3339), tenant, source (firewall|crowdstrike|aws|m365|ad|api|network), vendor,
product, event_type, event_subtype, severity (0-10), action
(allow|deny|create|delete|login|logout|alert), src_ip, src_port, dst_ip, dst_port, protocol, user,
host, process, url, http_method, status_code, rule_name, rule_id, cloud.account_id, cloud.region,
cloud.service, raw (original message), _tags (array)
```

---

4) Short Sample Logs for Testing

These examples may be used as seed data or submitted through the API.

4.1 Firewall/Syslog

```
<134>Aug 20 12:44:56 fw01 vendor=demo product=ngfw action=deny src=10.0.1.10 dst=8.8.8.8 spt=5353
dpt=53 proto=udp msg=DNS blocked policy=Block-DNS
```

4.2 Network (Router Syslog)

```
<190>Aug 20 13:01:02 r1 if=ge-0/0/1 event=link-down mac=aa:bb:cc:dd:ee:ff reason=carrier-loss
```

4.3 HTTP API (POST /ingest)

```
{
  "tenant": "demoA",
  "source": "api",
  "event_type": "app_login_failed",
  "user": "alice",
  "ip": "203.0.113.7",
  "reason": "wrong_password",
  "@timestamp": "2025-08-20T07:20:00Z"
}
```

4.4 CrowdStrike (sample JSON)

```
{
  "tenant": "demoA",
  "source": "crowdstrike",
  "event_type": "malware_detected",
  "host": "WIN10-01",
  "process": "powershell.exe",
  "severity": 8,
  "sha256": "abc...",
  "action": "quarantine",
  "@timestamp": "2025-08-20T08:00:00Z"
}
```

4.5 AWS CloudTrail (abridged)

```
{
  "tenant": "demoB",
  "source": "aws",
  "cloud": {"service": "iam", "account_id": "123456789012", "region": "ap-southeast-1"},
  "event_type": "CreateUser",
  "user": "admin",
  "@timestamp": "2025-08-20T09:10:00Z",
  "raw": {"eventName": "CreateUser", "requestParameters": {"userName": "temp-user"}}
}
```

4.6 Microsoft 365 Audit (abridged)

```
{
  "tenant": "demoB",
  "source": "m365",
  "event_type": "UserLoggedIn",
  "user": "bob@demo.local",
  "ip": "198.51.100.23",
  "status": "Success",
  "workload": "Exchange",
  "@timestamp": "2025-08-20T10:05:00Z"
}
```

4.7 Microsoft AD/Windows Security (abridged, Event ID 4625)

```
{
  "tenant": "demoA",
  "source": "ad",
  "event_id": 4625,
  "event_type": "LogonFailed",
  "user": "demo\\eve",
  "host": "DC01",
  "ip": "203.0.113.77",
  "logon_type": 3,
  "@timestamp": "2025-08-20T11:11:11Z"
}
```

5) Reference Architecture

Candidates may select, modify, or replace the following components:

- Collector/Ingest: Vector / Fluent Bit / Logstash / Custom (Node/Go/Python)
- Storage/Index: OpenSearch / ClickHouse / PostgreSQL (+JSONB/GIN) / Elasticsearch
- Backend API: FastAPI / Express / Go Fiber / NestJS (with authentication and an ingest endpoint)
- UI/Dashboard: React/Vue + chart library, or OpenSearch Dashboards/Grafana
- Packaging: Docker Compose (Appliance) + Cloud VM/Container (SaaS)

Minimum appliance requirements: Ubuntu 22.04+, 4 vCPU, 8 GB RAM, 40 GB disk, with the necessary ports open (e.g., 80/443/514).

6) Deliverables

1. A Git repository containing:
   - /docs/architecture.md (diagram + explanation of the data flow and tenant model)
   - /docs/setup_appliance.md and /docs/setup_saas.md (detailed installation steps)
   - docker-compose.yml (and/or Helm chart) + initialization/seed scripts
   - .env.example (required settings), Makefile/run.sh script
   - /samples/ (sample log files + sender scripts such as send_syslog.sh and post_logs.py)
   - /backend/, /frontend/, /ingest/ source code with README
   - /tests/ (at least 2-3 example test cases)
2. 30-minute demo video: Explain the architecture and demonstrate ingest -> search -> dashboard -> alert.
3. Demo URL (SaaS) and/or an OVA file or instructions for spinning up the appliance.
4. Postman/Insomnia collection for the ingest/search APIs, if applicable.

7) Acceptance Checklist

- Start the system in Appliance mode by following the documentation (one command or only a few steps).
- Send a sample Syslog message (e.g., using logger/nc) and see it in the UI within one minute.
- Call POST /ingest with the sample JSON and successfully search for the record.
- Upload or point the system to sample AWS/M365/AD files and verify that they are normalized.
- Verify that Top N, Timeline, and tenant/source/time filters work on the dashboard.
- Create a sample alert rule and observe a notification in the UI, email, or webhook.
- Test RBAC: a Viewer can see only their own tenant.
- Access the SaaS deployment over HTTPS.

---

8) Scoring Criteria (100 points)

Category	Details	Points
Architecture & Documentation	Clarity, design, and rationale for technology choices	15
Ingestion	Multiple sources/protocols, stability, and simulation	20
Normalization/Schema	Appropriate schema and mapping design	10
Storage & Query	Fast/accurate search and good indexing/partitioning	10
Dashboard/UI	Ease of use, with required charts/tables/filters	10
Alerting	At least one rule with successful notification	10
Security	Minimum AuthN/AuthZ, RBAC, and TLS	10
Deployment	Working Appliance + SaaS deployments	10
Tests & DX	Test examples, scripts/Makefile, and .env.example	5
Total		**100**

Bonus points (maximum +10): True multi-tenancy (separate index/table), enrichment, CI/CD, IaC, observability (metrics/traces), and hardening.

Minimum passing score: 60 points

Strong Hire consideration: 85 points or higher, with the ability to clearly explain the design rationale.